

Tree



Dr. Md. Atiqur Rahman
Assistant Professor
Dept. of Computer Science & Engineering

Tree

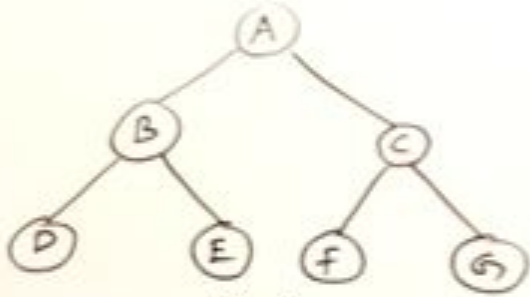


Fig-1

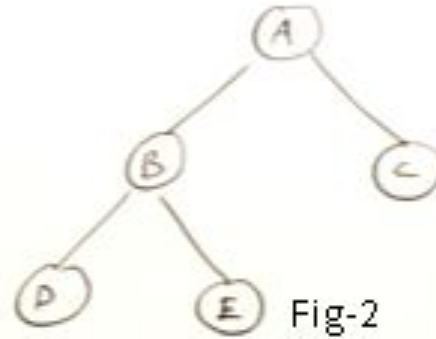


Fig-2

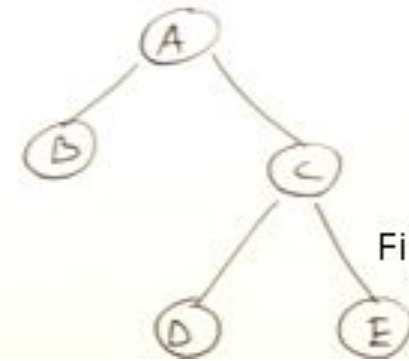
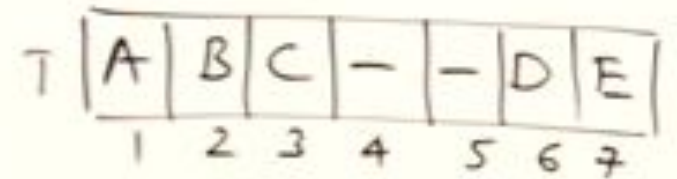
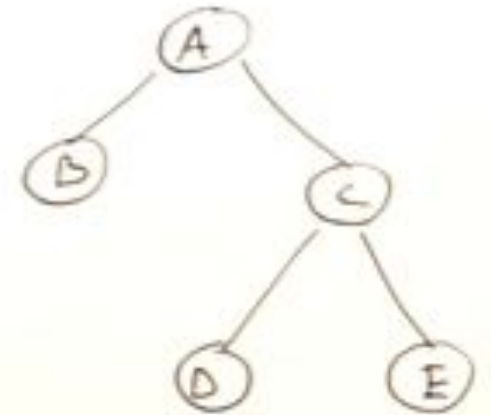
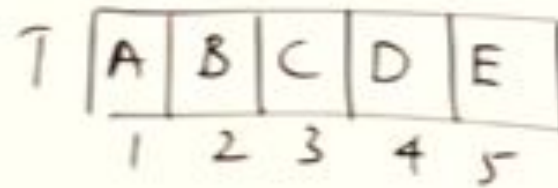
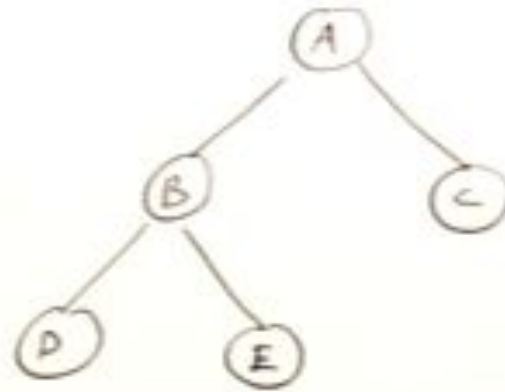
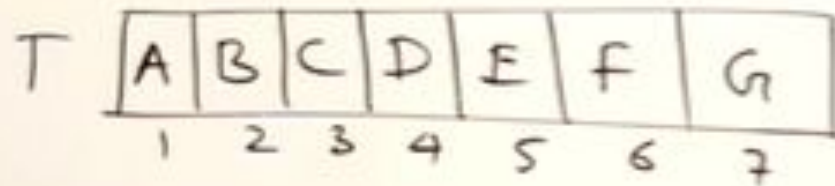
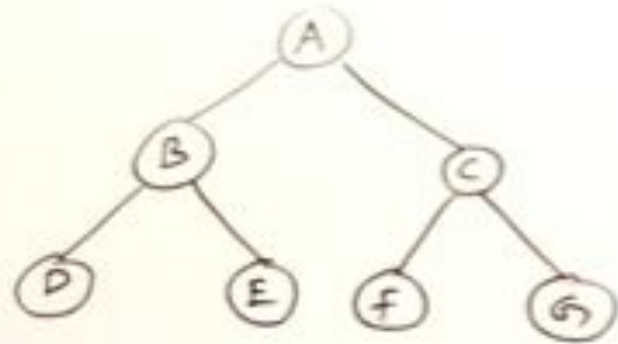


Fig-3

	Binary Tree	Full binary tree	Complete binary tree
Fig-1	Yes	Yes	Yes
Fig-2	Yes	No	Yes
Fig-3	Yes	No	No

Tree



if a Node is at index i -

its left child is at $2*i$

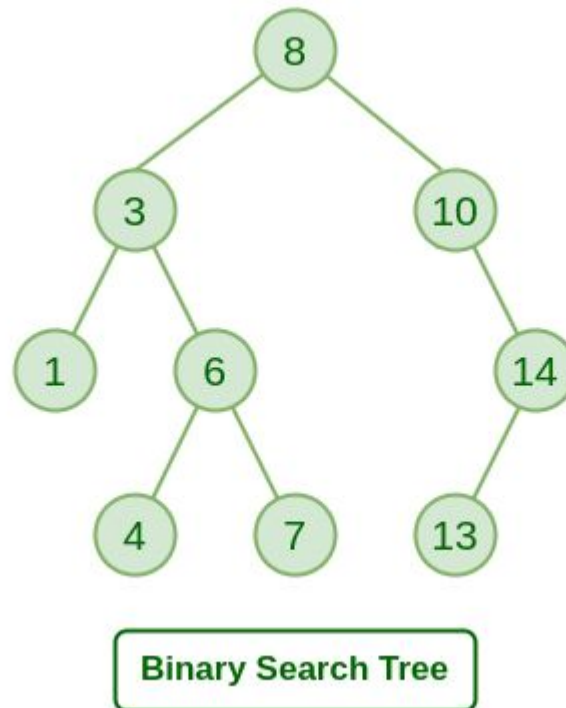
its right child is at $2*i+1$

its parent is at $\left\lfloor \frac{i}{2} \right\rfloor$

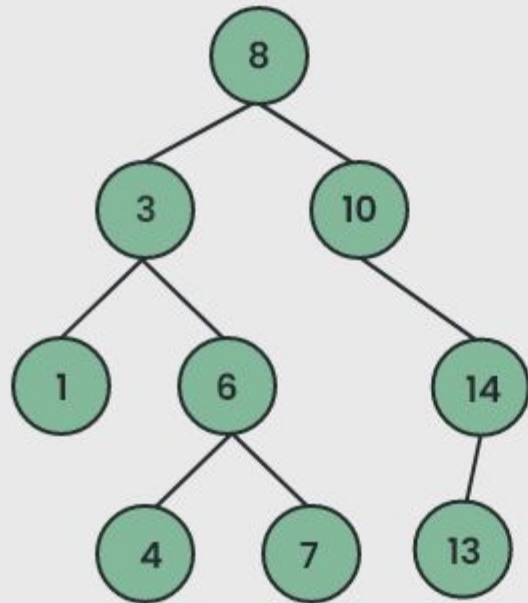
Binary Search Tree

A **Binary Search Tree** is a data structure where each node has at most two children: a left child and a right child. The left child contains values less than the parent node, and the right child contains values greater than the parent node.

Draw a binary search tree by inserting the numbers from left to right: 8, 3, 10, 1, 6, 14, 4, 7, 13

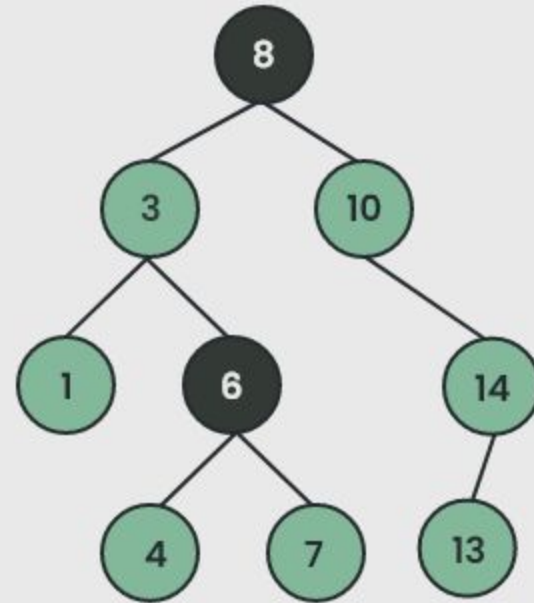


Searching in Binary Search Tree (BST)



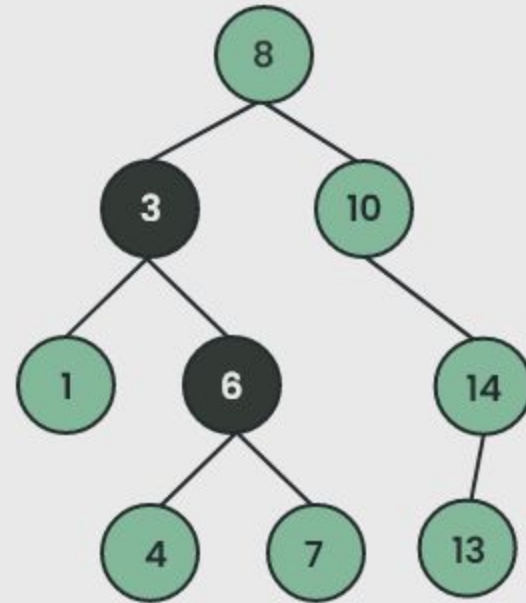
Consider The Following BST
Key = 6

Searching in Binary Search Tree (BST)



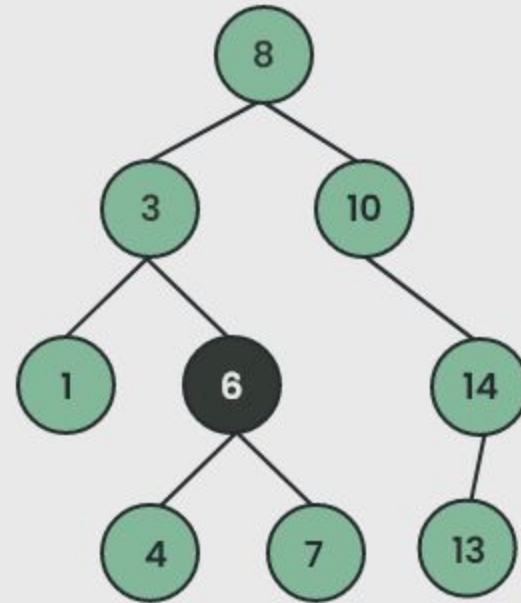
Compare Key With Root, i.e 8
as $6 < 8$, search in left subtree
of 8

Searching in Binary Search Tree (BST)



As Key (6) Is Greater Than 3,
Search In The Right Subtree Of 3

Searching in Binary Search Tree (BST)



As 6 Is Equal To Key (6), So We Have Found The Key

Searching in Binary Search Tree (BST)

The time complexity for searching an element in a binary search tree (BST) depends on the structure of the tree.

Here are the time complexities:

1. Average-case time complexity: $O(\log n)$

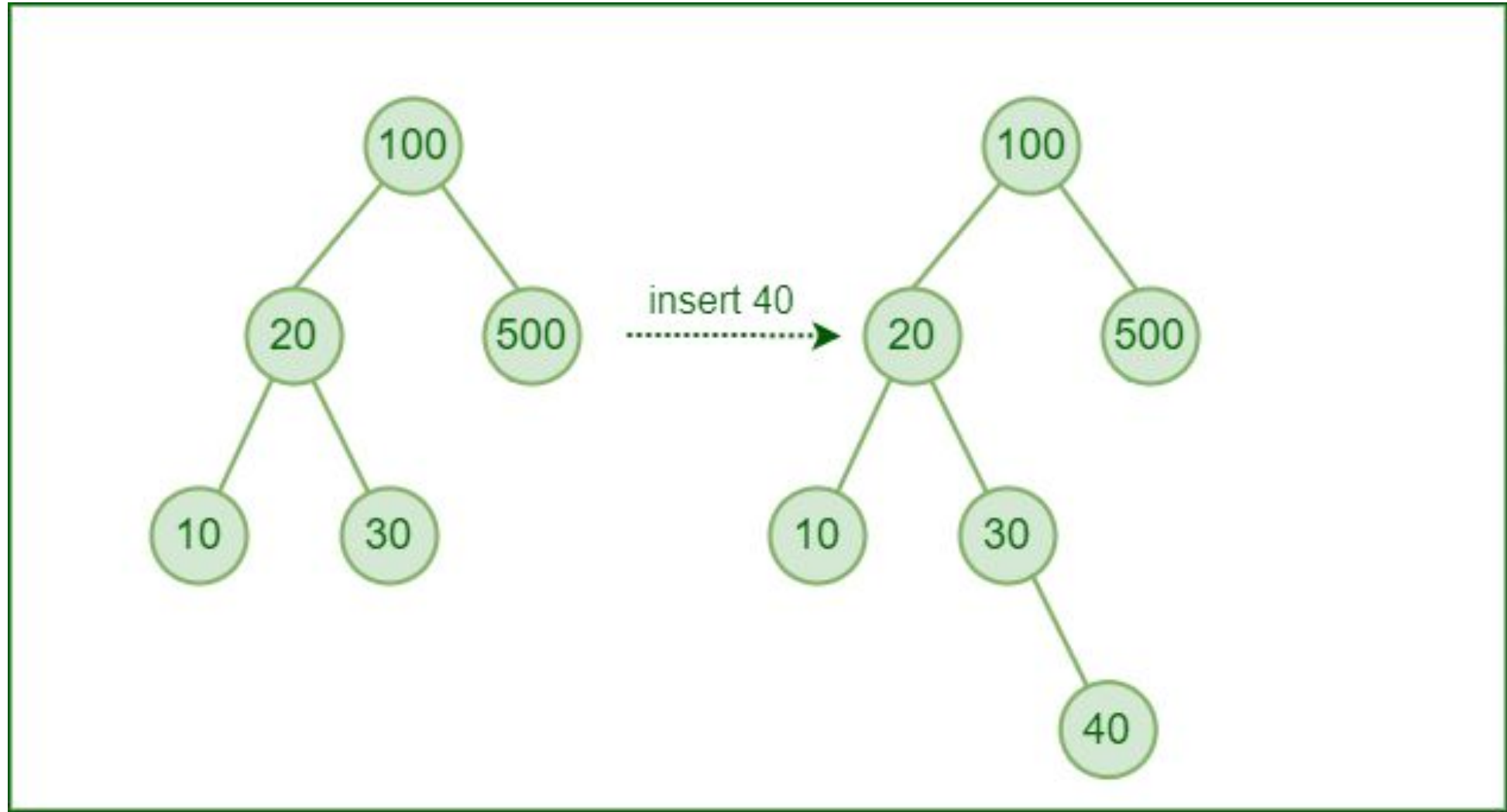
1. In a balanced BST, where the tree is well-balanced, the average-case time complexity for searching an element is $O(\log n)$, where n is the number of nodes in the tree. This is because at each step of the search, the number of remaining nodes to be searched is halved, leading to a logarithmic time complexity.

2. Worst-case time complexity: $O(n)$

1. In an unbalanced BST, the worst-case time complexity for searching is $O(n)$, where n is the number of nodes in the tree. This occurs when the tree is skewed heavily to one side, and each search operation essentially traverses through all nodes in the tree.

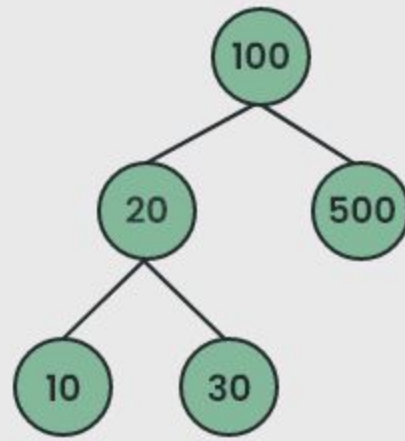
Space complexity : The space complexity of a binary search tree is $O(n)$.

Insertion in Binary Search Tree (BST)



Insertion in Binary Search Tree (BST)

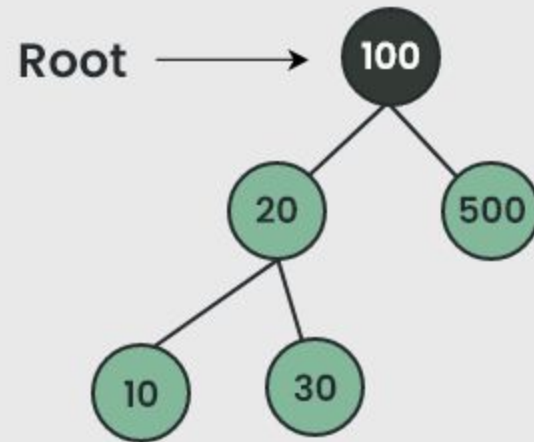
Consider The Following BST



X = 40 (The Node To Be Inserted)

Insertion in Binary Search Tree (BST)

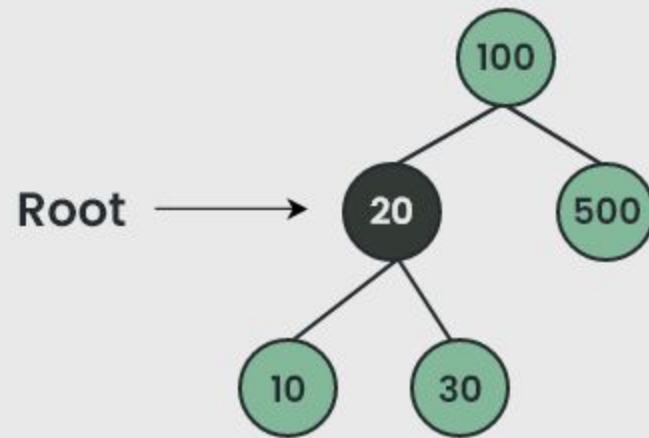
STEP 1 : Comparing X with Root Node



Since 100 Is Greater Than 40.
Move Pointer To The Left Child (20)

Insertion in Binary Search Tree (BST)

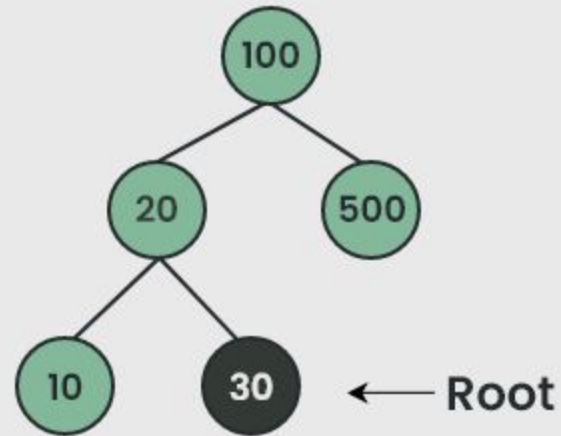
STEP 2 : Comparing X with left child of root node



Since 20 Is Less Than 40, Move
Pointer To The Right Child (30)

Insertion in Binary Search Tree (BST)

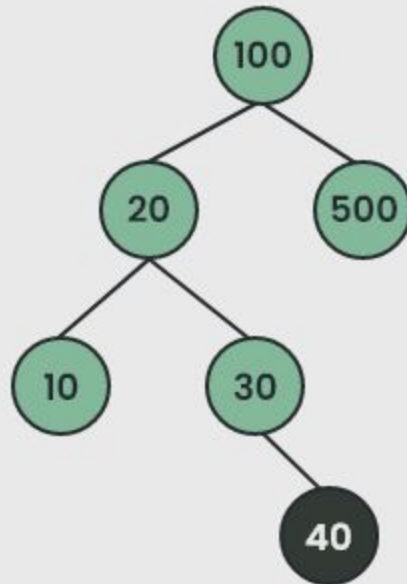
STEP 3 : Comparing x with the right child of 20



Again 40 Is Greater Than 30
Move Pointer To The Right Side
Of 30

Insertion in Binary Search Tree (BST)

STEP 4 :Insert item to the right of 30

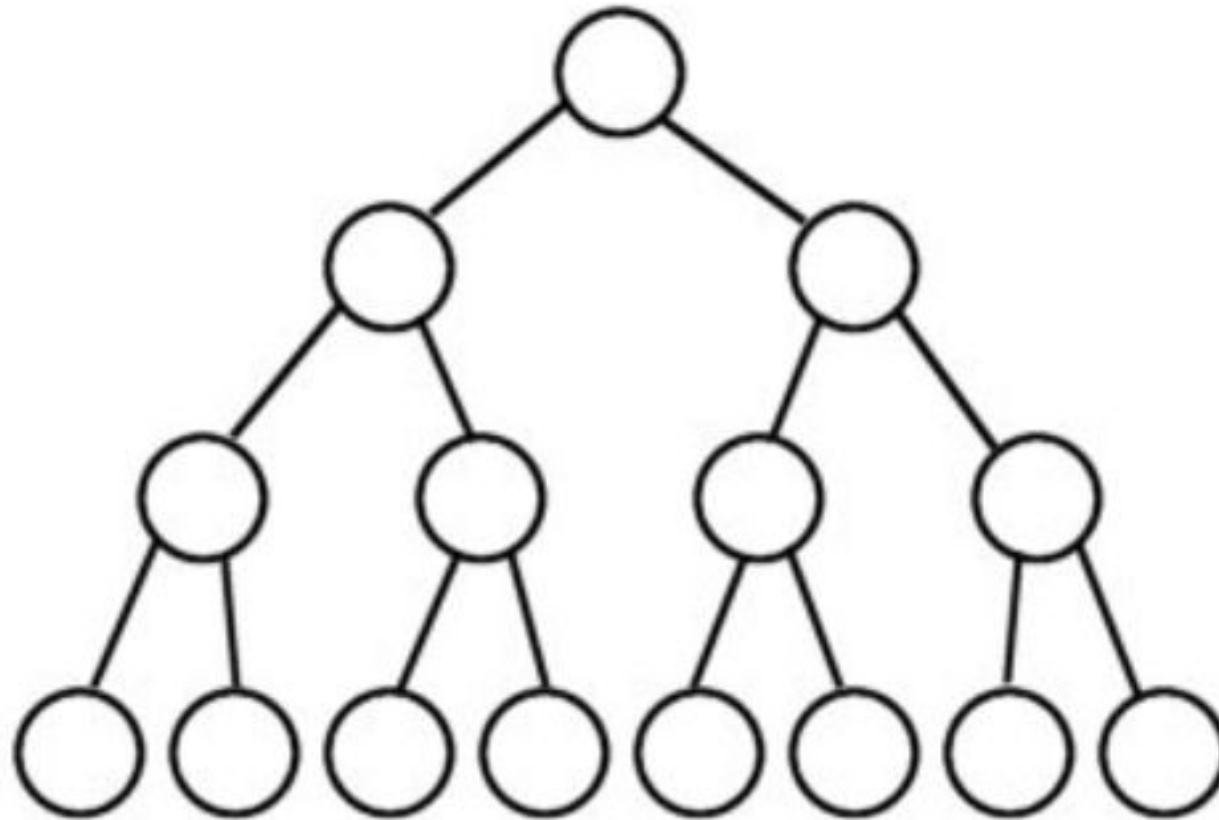


As 40 Is Greater Than The Node 30,
Thus It Will Be Inserted To The Right
Of 30

← Inserted
Node

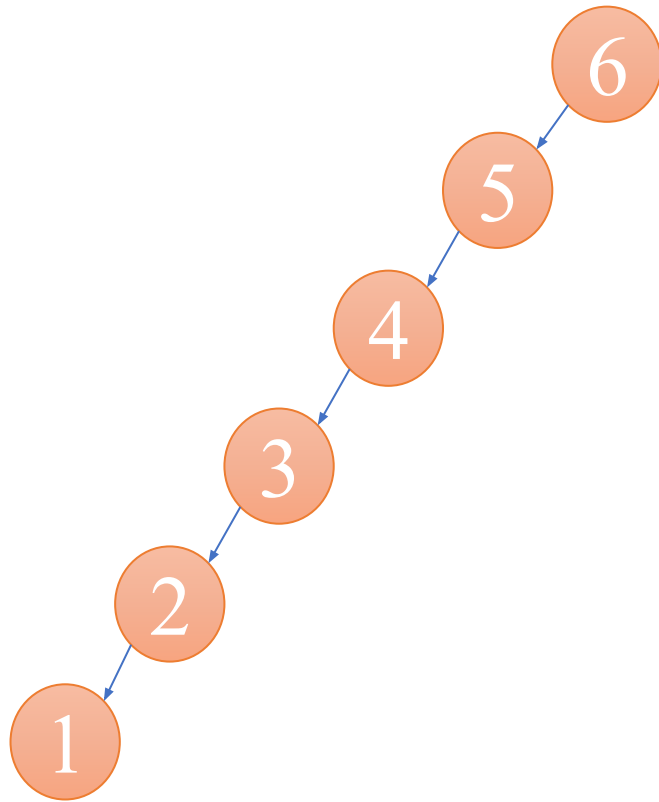
Time complexity for Inserting in Binary Search Tree (BST)

When a binary search tree (BST) is a complete binary tree, full binary tree, or a balanced tree, then the best-case time complexity occurs, and the best-case time complexity is $O(\log n)$ where $\log n$ is the height of the tree.



Time complexity for Inserting in Binary Search Tree (BST)

Draw a binary search tree by inserting the numbers from left to right: 6, 5, 4, 3, 2, 1



The height of the BST is equal to the number of elements, and the worst-case time complexity occurs in this scenario, with a time complexity of $O(n)$.

Space complexity : The space complexity of a binary search tree is $O(n)$.

Deletion in Binary Search Tree (BST)

Deleting a node from a binary search tree can be broken down into 3 scenarios.

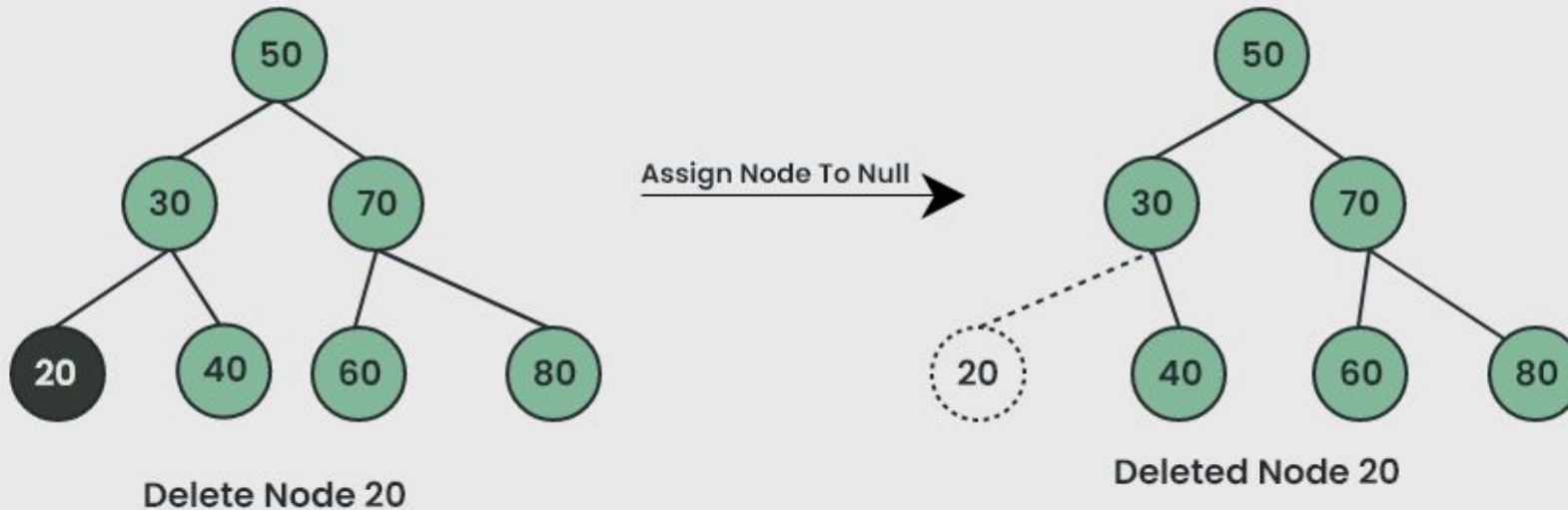
1. Delete a Leaf Node in BST
2. Delete a Node with Single Child in BST
3. Delete a Node with Both Children in BST
 - I. In order predecessor
 - II. In order successor

Deletion in Binary Search Tree (BST)

Delete a Leaf Node in BST

20 has been deleted from the following BST.

Case 1 : Delete A Leaf Node In BST

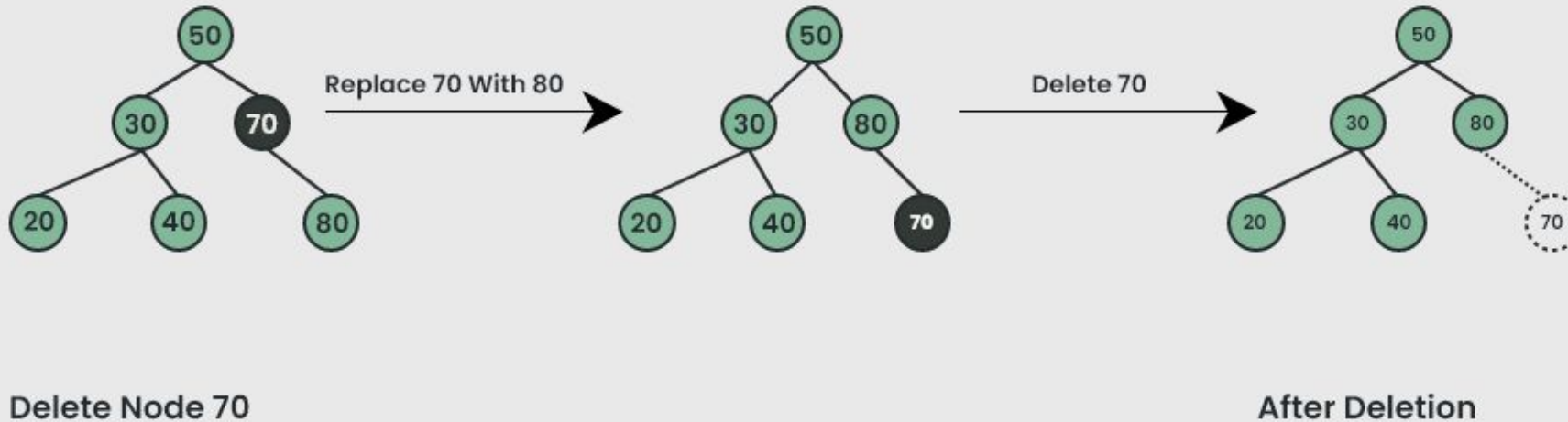


Deletion in Binary Search Tree (BST)

Delete a Node with Single Child in BST

70 has been deleted from the following BST.

Case 2: Delete A Node With Single Child In BST

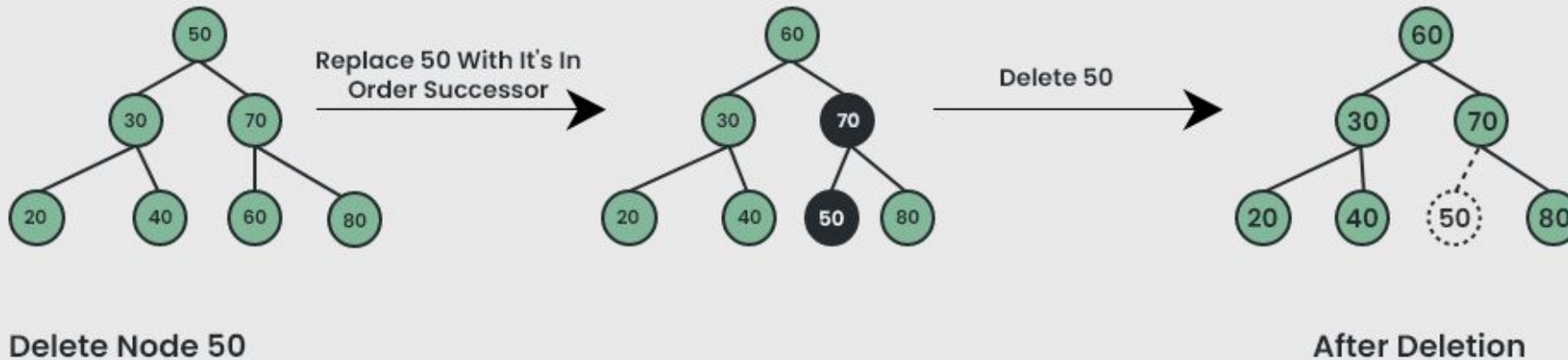


Deletion in Binary Search Tree (BST)

Delete a Node with Both Children in BST

One of the two methods (In-order predecessor or In-order successor) can be used to delete a node from a BST in this case. The In-order successor is used to delete the node 50 from the following BST.

Case 3 : Delete A Node With Both Children In BST



Time complexity for Deleting a node from a Binary Search Tree (BST)

The time complexities for deleting an element from a binary search tree (BST) depend on various factors, such as the structure of the tree and the specific node being deleted. Here are the average-case and worst-case time complexities for different scenarios:

1. Average-case time complexity:

1. Balanced Tree: $O(\log n)$
2. In a balanced BST, the average-case time complexity for deletion is $O(\log n)$, where n is the number of nodes in the tree.

2. Worst-case time complexity:

1. Skewed Tree: $O(n)$
2. If the binary search tree is heavily skewed (e.g., formed in the shape of a linked list), the worst-case time complexity for deletion can be $O(n)$, where n is the number of nodes in the tree.

Space complexity : The space complexity of a binary search tree is $O(n)$.

Thank You